HoTEE: Building Secure and Logarithmic-complexity Search Indexes for Hybrid-trust Encrypted Databases

Abstract

Place holder. Driven by the rapid advancement of cloud computing, organizations are increasingly moving their sensitive data to cloud services (e.g., databases), which offers elasticity, high availability, and low cost. For example, the US state and local governments use AWS to host databases for criminal records [1]; the Harvard global health institute uses Google cloud to store healthcare data (e.g., hospitalizations, ventilator utilization) in databases for COVID-19 forecast [3]. Besides, to generate sufficient data insights and give precise analytics, organizations have grown increasingly collaborative to share their data and make joint queries on all databases [36, 43].

Place holder. Driven by the rapid advancement of cloud computing, organizations are increasingly moving their sensitive data to cloud services (e.g., databases), which offers elasticity, high availability, and low cost. For example, the US state and local governments use AWS to host databases for criminal records [1]; the Harvard global health institute uses Google cloud to store healthcare data (e.g., hospitalizations, ventilator utilization) in databases for COVID-19 forecast [3]. Besides, to generate sufficient data insights and give precise analytics, organizations have grown increasingly collaborative to share their data and make joint queries on all databases [36, 43]. Besides, to generate sufficient data insights and give precise analytics,

1 Introduction

Driven by the rapid advancement of cloud computing, organizations are increasingly moving their sensitive data to cloud services (e.g., databases), which offer elasticity, high availability, and low cost. For example, the US state and local governments use AWS to host databases for criminal records [1]; the Harvard global health institute uses Google cloud to store healthcare data (e.g., hospitalizations, ventilator utilization) in databases for COVID-19 forecast [3]. Besides, to generate sufficient data insights and give precise analytics, organizations have grown increasingly collaborative to share their data and make joint queries on all databases [36, 43].

Owing to the untrusted nature of the cloud [40], organizations typically require their uploaded data in cloud databases to keep confidential from adversaries, including both the insiders with administrative privileges and attackers who compromise the cloud database services [2, 38]. Existing sophisticated databases (e.g., MySQL) employ access control and data-at-rest encryption [31], which can be easily bypassed by adversaries and exfiltrate plaintexts during query execution.

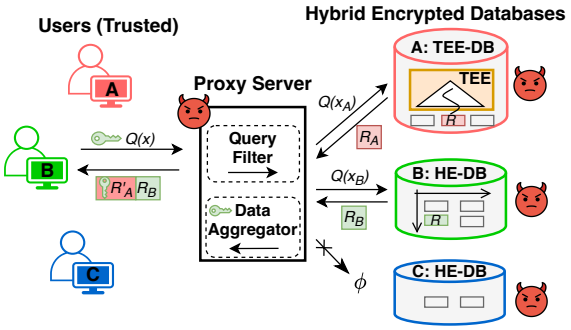


Figure 1: HoTEE's hybrid-trust multi-party joint query scenario.

To protect data confidentiality on cloud databases, both the industry and academia have expedited the developments of *encrypted databases* [2, 17, 20, 29, 32, 34, 38, 41], and existing work can be divided in two categories. The first category (e.g., EnclaveDB [34], Azure EdgelessDB [2], Enclage [38]) uses Trusted Execution Environment (TEE), in particular Intel SGX [15], to execute queries on plaintexts in a hardware-level isolated region called enclave [15]. *TEE databases* can operate on plaintexts with high efficiency, but sustain the risk of trusting hardware vendors and long-lasting debates on side-channel attacks [22] that deduce plaintexts at runtime.

The second line of work (e.g., CryptDB [32], HElibDB [4], FHEDB [29]) relies on homomorphic encryption (HE), especially fully HE schemes (e.g., BGV [18]), to allow databases compute arbitrary aggregations over homomorphic data, and only the users can decrypt to see plaintext results. With a cryptographic root-of-trust, *HE databases* provide stronger confidentiality guarantees than TEE databases, hence attracting an increasing number of organizations. For instance, Canada government has planned to link confidential data dispersed across the government departments and public institutions using HE databases [44].

Since different organizations can have different trusts and preferences in encrypted databases, motivated by the growing demand for multi-party collaborative analytics [41, 43], we consider a hybrid-trust multi-party joint query scenario wherein a group of authenticated parties shares sensitive data stored in hybrid encrypted databases (i.e., either a TEE or HE database) on the cloud and all databases jointly answer queries from one of the parties (Figure 1). Specifically, queries are first filtered and forwarded to relevant databases in which the queried content exists, then databases execute searches on encrypted data to produce partial results, and finally all partial results are securely aggregated and returned to the querier. Such a computing model can support a wide range of secure

applications. For instance, physicians can make distributed queries to multiple distinct medical institutions owning sensitive symptom records for collaborative diagnosis.

Unfortunately, to the best of our knowledge, despite much academic and industrial effort in developing systems for distributed joint queries on encrypted databases [9, 41, 43], none of them are designed for the hybrid-trust scenario where TEE and HE databases coexist. This is due to two major challenges that stem from the utility gap between TEE and HE databases.

First, as TEE databases directly operate on plaintexts in enclaves, so they support all typical query operations [27], including *equality*, *range*, *aggregation*, or their combinations, by searching on well-studied *logarithmic-complexity* indexes (typically, a B^+-tree) in enclaves. However, none of existing HE databases can efficiently support all these operations, bringing a cascading effect of limiting TEE databases' utility in the hybrid-trust joint query scenario.

Specifically, existing HE databases use two methods to support operations other than HE-enabled aggregation locally. One line of work (e.g., HElibDB [4], FHEDB [29]) uses specific cryptographic functions (e.g., FLT [4]) to evaluate the equality of homomorphic data by scanning the whole dataset with *linear-complexity*, which is inefficient given massive (and constantly growing) amounts of outsourced data and does not support range search; another line of work (e.g., CryptDB [32]) uses distinct encryption methods to support either operations, i.e., use deterministic encryption [10] for equality search and order-preserving encryption [5] for range search with logarithmic-complexity , but only supports static data with known distribution (i.e., write-limited) and cannot support both operations simultaneously.

The second challenge is to enable the proxy to securely exclude non-matching data in irrelevant distributed encrypted databases. Specifically, in traditional plaintext distributed databases (e.g., BigTable [13], PostgreSQL [33]), queries are first committed to an index server (i.e., proxy) equipped with a membership index, which analyzes query conditions to early-exclude non-matching data in irrelevant databases before query execution to reduce unnecessary I/O operations and accelerate distributed query processing.

Nevertheless, a new confidentiality-locality gap arises when querying distributed encrypted databases: organizations' sensitive data must be kept confidential from a third-party proxy, and the proxy should locate encrypted databases that comprise matching data. Although existing systems [11, 17] propose to use Bloom filter [12], a space-efficient index to check the membership of queried data in encrypted databases, however, Bloom filters only support equality checks, and cannot handle multi-party data membership checking in the hybrid-trust joint query scenario.

In this paper, we propose HoTEE[1], the first hybrid-trust joint query framework that efficiently performs all typical query operations (equality, range, aggregation) in heterogeneous encrypted databases while ensuring end-to-end data confidentiality. To tackle the aforementioned challenges, we present a new *stealth index* abstraction called *MeRF-Tree*, which supports logarithmic-complexity indexing for equality and range searches on encrypted data simultaneously and enables non-matching data exclusion for general queries, solving two challenges with one abstraction in a unified manner.

To build MeRF-Tree stealth index, our key insight is that filters have an intrinsic stealth feature of hiding plaintexts, e.g., Bloom filter maps plaintexts to random positions in distinct bitmaps as `"1"`, checks membership by verifying whether every position for queried data is `"1"`. Inspired by filter's bitwise stealth feature, we can stack filters into a Merkle-tree style data structure where each node's filter is the *OR* gate computation result of its child nodes' filters, such that the resultant tree can support logarithmic-complexity indexing for equality search without revealing plaintexts. Furthermore, to support stealth range search, we propose *stealth bias* to patch a new range property to index nodes' filters, such that filters can preserve the same numerical orders as their corresponding plaintexts. Consequently, we term the stealth index as Merkle Range Filter Tree (i.e., MeRF-Tree).

Unfortunately, even with the stealth index, another security problem arises when aggregating multi-party data at proxy, given that the retrieved data may stem from different organizations and is encrypted under distinct keys which cannot be directly shared. A trivial approach is to set up TEE enclaves at the proxy, transform all retrieved data by decrypting and then re-encrypting with the querier's key pre-stored in proxy enclaves. However, owing to different trusts of organizations, the retrieved homomorphic data from HE databases is forbidden to decrypt as plaintexts in enclaves (§2.2), hence cannot be transformed to querier-decryptable data in proxy enclaves.

To remedy this issue, we identified a cryptographic tool, *key switch* [18] (an algorithm for reducing homomorphic data noises), which allows us to securely switch encryption keys of homomorphic data without revealing plaintexts. Based on the tool, HoTEE designs a secure distributed key switch protocol which securely transforms multi-party data for aggregation without breaking the hybrid trusts, and protects data confidentiality even when proxy and querier collude (§4).

We implemented HoTEE on CryptDB codebase [32], a mature modular framework for evaluating distributed encrypted databases. We built two HE databases with HoTEE by augmenting prominent databases: HoTEE-MySQL and HoTEE-WiredTiger. MySQL [30] is the most widely used SQL backend for encrypted databases; WiredTiger [14] is a popular key-value store that is the default NoSQL backend for MongoDB. We run HE database with an open-source TEE database Azure EdgelessDB [2] for hybrid-trust joint queries. We compared HoTEE with two famous HE databases CryptDB [32] and HElibDB [4] using typical workloads (e.g., TPC-C [25]) evaluated in relevant systems [6, 32, 42]. Evaluation shows that:

[1] HoTEE stands for Homomorphic-TEE hybrid-trust joint query.

	HoTEE	MySQL [30]	EnclaveDB [34]	EdgelessDB [2]	Cryptdb [32]	HElibDB [20]	FHEDB [29]	HybrTC [41]	Dory [17]
Data & query confidentiality	✓	✗	✓	✓	✓	✓	✓	✓	✓
Hybrid-trust joint query	✓	✗	✗	✗	✗	✗	✗	✓°	✓°
Equality search	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$	n/a.	$O(n)$
Range search	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	n/a.	n/a.	n/a.	n/a.
Insertion	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	$O(1)$	$O(1)$	n/a.	$O(1)$

°: Potentially support, but not explicitly handle hybrid-trust joint queries.

Table 1: Comparison of HoTEE and existing systems. "**n/a**." means not applicable as they do not support such query operations.

- **Utility**. HoTEE is the first generic framework that enables distributed queries on multi-party encrypted databases, supports general queries with logarithmic search complexity and preserves end-to-end confidentiality of all parties' data in a hybrid-trust computing scenario (§7).
- **Efficiency**. HoTEE-MySQL achieved xx% and xx% lower latency on average for equality search and range search than baselines; HoTEE-WiredTiger scaled out to larger datasets and more parties with almost linearly increased throughput while querying latency remained stable (§7).
- **Extensibility**. HoTEE supports various types of databases, including both relational and NoSQL databases (e.g., MySQL and WiredTiger).

Our main contribution is MeRF-Tree, a new stealth index abstraction that supports logarithmic-complexity indexing for general search queries (e.g., equality and range) on distributed encrypted databases without revealing plaintexts. This enables HoTEE to securely bridge the utility gap between heterogeneous encrypted databases, and allows early exclusion of non-matching data under various search conditions. Our other contributions include a secure distributed key switch protocol, which is the first effective attempt to tackle hybrid-trust data aggregation, making HoTEE unique to facilitate multiple parties with distinct trusts on encrypted databases to securely share data and make joint queries in various mission-critical applications (e.g., medical diagnosis [24, 26], governmental data collection [9, 44]). HoTEE's source code is released on `github.com/osdipxxx/HoTEE`.

In the rest of this paper: §2 introduces background; §3 gives an overview of HoTEE; §4 describes HoTEE's design; §6 covers HoTEE's implementation; §7 shows our evaluation; §8 discusses related work and §9 concludes.

2 Background and Motivation

2.1 Securing Data for Cloud Databases

Today, organizations are increasingly moving their constantly growing amounts of data to cloud databases [1, 3]. Yet, many organizations still decline to widely adopt cloud databases for data management due to severe confidentiality concerns. To address the security issue, techniques have emerged that allow queries to be run against encrypted data kept in cloud databases. Two prominent approaches consist of hardware-based solutions using *Trusted Execution Environment* (TEE) like Intel SGX [15] and software-based solutions using *Homomorphic Encryption* (HE) like BGV [18].

Intel SGX. SGX [15] is a pervasively used hardware feature on commodity CPUs. SGX provides a secure execution environment called *enclave*, where data and code execution cannot be seen or tampered with from outside. SGX has been widely adopted in recent encrypted databases [2, 34, 38] because (1) SGX provides sophisticated API including sealing and attestation, and (2) the latest SGX 2.0 greatly expands enclave memory capacity (up to 512GB/CPU [21]) to fit memory-intensive applications including databases [38].

Homomorphic BGV. To avoid trusting one TEE hardware vendor, some organizations are seeking a cryptographic root-of-trust from homomorphic encryption (HE), and are willing to pay for the deficiencies of using HE-based solutions, especially fully HE schemes like BGV [18]. BGV allows arbitrary aggregation (e.g., `sum`, `avg`) on encrypted data (with noises) without requiring a secret key for decryption, and the computation result can be fully revealed by the owner of the secret key [18]. BGV uses *key switch* algorithm to reduce ciphertext's noises, which is leveraged by HoTEE to aggregate multi-party data encrypted by distinct secret keys (§4.2).

2.2 Heterogeneous Encrypted Databases

To make a clear comparison, we summarize the representative systems and compile their features in Table 1. Note that, the traditional MySQL database uses access control and data-at-rest encryption to protect data [31], failing to provide data and query confidentiality (i.e., process encrypted queries on the encrypted data to defend against adversaries described in §1), but enjoys full-fledged search functionalities with logarithmic-complexity indexing on plaintexts using B^+-tree [30].

TEE databases. The first category is TEE-based encrypted databases. EnclaveDB [34] makes user data completely inaccessible by implementing in-memory storage and query engine in enclaves, protecting data and query confidentiality and supporting logarithmic-complexity operations using a TEE-shielded B^+-tree. Azure EdgelessDB [2], and Alibaba Enclage [38] consider on-disk storage (sealed by SGX) and use B^+-tree for efficiently indexing as well: Enclage shields

most B$^+$-tree nodes in plaintext within enclaves; EdgelessDB stores an AES-encrypted B$^+$-tree outside enclaves, and performs searches by on-demand pulling index nodes to enclaves and then decrypting and comparing each node for indexing.

HE databases. Different from TEE databases that decrypt and then search plaintexts on a fully (or partially) shielded B$^+$-tree in enclaves, the homomorphic data is forbidden to decrypt and search in enclaves because HE database users demand a cryptographic root-of-trust thus distrust TEEs (§2.1).

To build search indexes for HE databases, CryptDB [32] uses deterministic encryption [10] for equality search by linearly scanning all data and uses order-preserving encryption [5] (OPE) for range search. Since OPE can preserve the original orders of plaintexts, one can use off-the-shelf indexes like B$^+$-tree for OPE ciphertexts same as plaintexts. However, OPE does not support equality search, and CryptDB's solution (i.e., onion encryption) becomes inefficient when the workload is mixed [39]. Besides, OPE assumes the plaintext domain distribution is fixed [28], which contradicts the dynamic nature of database writes. HElibDB [4] and FHEDB [29] use specific cryptographic tools to directly perform equality searches on homomorphic data with linear-complexity but lack support for range search on homomorphic data.

Hybrid-trust joint query. The aforementioned systems are designed for a single machine and do not support distributed queries on multiple hybrid-trust databases. HybrTC [41] integrates TEE with MPC [16] to support multi-party data analytics wherein users only trust their own TEE databases, but supports neither equality nor search queries on HE databases. Dory [17] supports only equality search for encrypted keywords by performing linear scans on bloom filters [12] because the bloom filter has limited expressiveness which tells only the existance of searched data on an encrypted database.

2.3 Motivation and Opportunities

From the survey results reported above, we summarize that different organizations can have different trusts and preferences in encrypted databases. Thus, in this paper, we envision the first hybrid-trust scenario where users make distributed queries to heterogeneous encrypted databases. Such a computing model brings two unique challenges. First, HoTEE should support general query operations with logarithmic search complexity on HE databases for joint query. Second, HoTEE should support encrypted query filtering and multi-party data aggregation same as distributed plaintext databases.

To tackle the challenges, we summarize what we considered opportunities to make the hybrid-trust joint query problem easier. First, we observe that filters used by traditional databases have an intrinsic stealth feature of hiding plaintexts while providing membership information, which can serve as a basic block to build an encrypted search index. Second, we can employ traditional homomorphic key switch algorithm to securely aggregate multi-party data, even if the algorithm has an orthogonal goal of reducing homomorphic data noises.

3 Overview

3.1 System Model

Entities. HoTEE's participants consist of database users (for short, users) and database servers (for short, servers). Users outsource their sensitive data to encrypted databases on cloud servers and later enjoy encrypted query service. For simplicity, we assume each user employs one server for storage. Same as traditional distributed databases [13, 33], users first submit queries to a proxy node (for short, proxy), which filters query conditions (encrypted) and re-directs queries to relevant servers to retrive encrypted data. The retrieved data is aggregated at the proxy and then returned to users.

Data model. Without losing generality, we use the relational data model as an example to illustrate our design in the following discussions. We will discuss the extensibility in §8.1.

Specifically, a table T contains n columns (or attributes) $CL = \{c_0, c_1, ..., c_{n-1}\}$ and has a primary key $PK \in CL$. Same as [34, 38], we consider the values in all columns are confidential, which means that each value should be encrypted before being sent to an encrypted database. For a record (i.e., a row) r, its value v_i in column c_i is ciphertext $E(v_i)$, where E represents the encryption scheme employed by the user: for TEE databases, E is typically AES GCM [23] and user's encryption key will be provisioned to SGX enclaves when computation over plaintext is needed; for HE databases, E is typically BGV [18], and the user never reveals his encryption key to the server or other users participated in joint queries.

Note that, same as existing encrypted databases [2, 34, 38], columns can be encrypted by distinct encryption keys to further strengthen confidentiality; it is also feasible to have plaintext columns if cross-column correlations are omitted [7, 8].

Storage model. Same as existing work [17, 38], in HoTEE, we focus only on the search system for end-to-end encrypted databases and not on the design of underlying storage engine. Many optimized storage engines [19, 37, 38] already exist and are used in encrypted databases. HoTEE is built on top of these storage engines, so below we abstract out functions that should be supported by an underlying storage engine.

Recall the aforementioned data model, we assume a table T with n columns $CL = \{c_0, c_1, ..., c_{n-1}\}$ and is indexed on the primary key c_0 (i.e., $PK = c_0$). A record r can be depicted as $[E(k), E(v_1), ..., E(v_{n-1})]$ where k is the value for c_0. The storage engine (SE) should provide the following API:

- EqualGet(E(k)): Retrieve the row r from T given a key k.
- RangeGet(E(k_s),E(k_e)): Retrieve all rows $\{r\}$ from T whose keys are between E(k_s) and E(k_e).
- FullGet(): Retrieve all rows $\{r\}$ from T.
- Put(r): Insert (or update) a row r to T if its key k is non-existent (or existent).
- Delete(E(k)): Delete the row r from T given a key k.

Note that, same as [2, 32, 38], HoTEE also supports searching on non-primary keys by building secondary indexes [35].

3.2 The HoTEE API

HoTEE provides a library called LibHoTEE, which contains basic API (Table 2) that facilitates heterogeneous encrypted databases to be deployed in HoTEE to jointly answer general queries. We will discuss how to integrate an encrypted database into HoTEE framework using LibHoTEE in §4.1. The data processing API consists of functions for users to submit general queries and for the proxy to process multi-party encrypted data (§4.2). The indexing API enables the proxy and servers to search encrypted data with HoTEE's stealth index abstraction: if called by the proxy, `stealthIdx(k)` returns *True* if the key k exists in a database; if called by the server, `stealthIdx(k)` returns the address of k in storage and calls the storage API (§3.1) to retrieve local data (§4.1).

3.3 Threat Model

Place holder.Three are three roles in our deployment scenario, namely, the data provider (e.g., hospital), the cloud storage provider (e.g., Amazon, Microsoft) and the data user (e.g., physician). The data provider and data user are trusted, providing that the data outsourced to the cloud storage is correct and users' queries are authenticated. Note that, we omit the authentication process here and assume all queries from users are authenticated by data providers through out-of-band services (e.g., xxx). We will have more query authentication discussions later (§**??**). Other roles are semi-honest: the cloud storage provider are curious about the plaintext of outsourced data, but will honestly execute queries to preserve reputation; the data users can query matching records but will learn nothing else stored in the cloud storage while preserving the query privacy.

We assume the proxy server is semi-honest. The proxy server will honestly execute data lookup and query routing, but is curious about the data confidentiality and query confidentiality. Such assumption requires the HoTEE's proxy server design to reveal data locality without leaking data confidentiality, and preserve query privacy while maintaining query functionality.

3.4 HoTEE's Workflow Overview

4 Protocol Description

4.1 The HoTEE Index

4.2 The HoTEE Workflow

5 Design

Why fundamental? Why groundbreaking? The first stealth index that (1) supports secure data look up and hybird query among TEE and Homomorphic databases, and (2) significantly improves Homomorphic database read bottleneck - by providing a stealth, fast and general ciphertext index.

From the strawman to the new design. The design of stealth index is full of fundamental research challenges. From the initial straw man design to our final design, we walk through these steps:

LibHoTEE API	Role	Description
Data Processing API		
❶ `newQry(COND)`	US[1]	Submit a new query with encrypted conditions (e.g., $E(k)$) to HoTEE's proxy (§4.1).
❷ `transform(E(K))`	PR[2]	Transform a key k to HoTEE's searchable index key (§4.1).
❸ `reqTok(Ks)`	US	Request new tokens from other users for key switch (§4.2).
❹ `genToK(Ks)`	US	Generate a new token tk using K_s for key switch (§4.2).
❺ `dSwitch(TK,D)`	PR	Run distributed key switch protocol to aggregate multi-party encrypted data (§4.2).
Indexing API		
❻ `stealthIdx(KTRAN)`	PR, SE[3]	Search a transformed key with HoTEE's stealth index (§4.1).

[1] **US**: USer [2] **PR**: PRoxy [3] **SE**: SErver

Table 2: The API provided by LibHoTEE.

(1) Use the stealth feature of bloom filter to securely store plaintext data on the server; use the existance feature of filter to make union of different filters, design **merkle bloom filter (MBF)** and design **MBF-Tree**. MBF-Tree is a bottom-up structure which indexs data through layers of merged bloom filters, enabling binary search with *logN*-level search time complexity. Unfortunately, MBF-Tree **does not support write and general range search** because (a) data stored in MBF-Tree are only filters, which can only be employed to do point matching and **no plaingtext order** information is revealed. Hence, newly added keys (identified by their filters) cannot find the proper position and add the new keys there. (If just append new keys, the index will no longer support range search). And (b), the current MBF-Tree does not support general range quries when the left/right boundary is a non-existent key. This is because the tree is just built on top of existing keys by merging there filters. Therefore, when searching a non-existent key, the MBF-Tree cannot find the right position through filter comparison.

(2) To support write and search on non-existent keys (**fundamental challenges**), we optimize the MBF-Tree with the following idea: add a new **range feature** on original bloom filter design, and use the distinct range information to confidentially sort the ciphertext in plaintext order and ensure even a non-existent key can use its range to find the proper position within ciphertexts. To make this happen, we use a **greedy bias filling** approach, which adds a larger bias to a bigger ciphertext while ensuring the monotonicity of ciphertexts. By using range as the tree traversal condition and filter and the match condition, we now support write because a newly added key (with its distinct filter and range) can be indexed to its proper position among ciphertexts, and support non-existent key search because all ciphertexts have their distinct range value, which indicates its plaintext order.

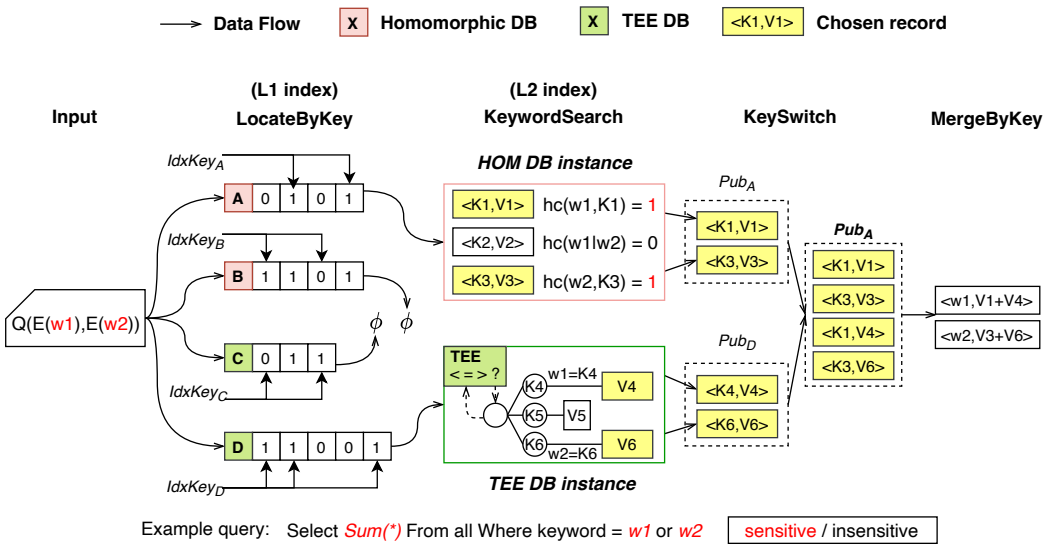


Figure 2: HoTEE workflow.

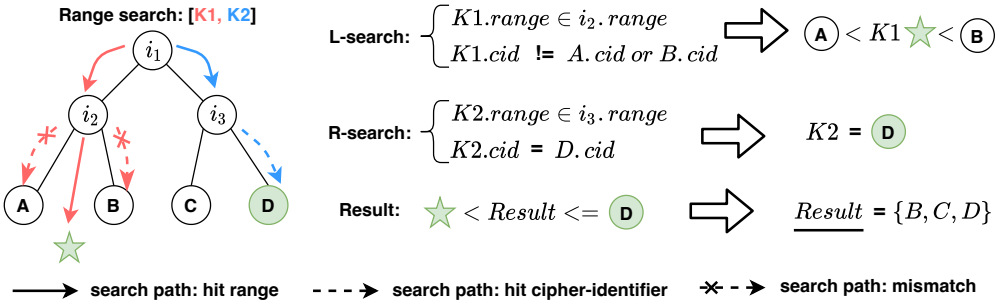


Figure 3: MBF-Tree stealth range search on non-existent key.

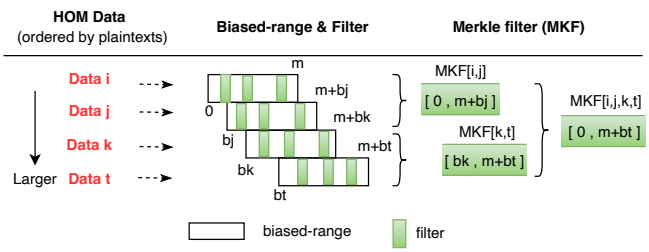


Figure 4: HoTEE index preparation.

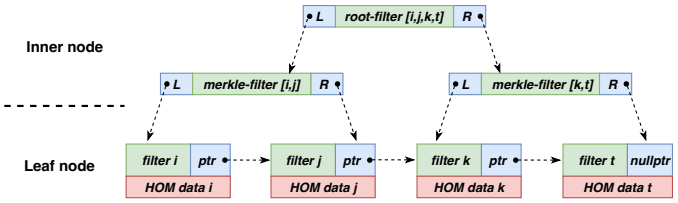


Figure 5: HoTEE's merkle filter tree.

(3) Finally, to reduce the storage overhead at client side, we make a **stateless design** to the bias function and propose a **stateless bias filling** technique. Concretely, **f(x)=g(x)+t(x,y,z)**, where g(x) is monotonically increasing, t(x,y,z) is a parameterized function, which has variation within a certain range and has a fixed number of values and does not overwhelm the monotonicity of g(x). With this design, now the client does not need to store all data to provide biases (e.g., f(x) = f(x-1)+ delta), the client only needs to store a bias function (e.g., f(x)= kx+hash(x), hash(x) ∈[0,k)) to produce biased range when adding a new key. (Note that, the filter itself is stateless as well because the client only needs several hash functions to produce a key's filters).

Threat model. Three are three roles in our deployment scenario, namely, the data provider (e.g., hospital), the cloud storage provider (e.g., Amazon, Microsoft) and the data user (e.g., physician). The data provider and data user are trusted, providing that the data outsourced to the cloud storage is correct and users' queries are authenticated. Note that, we omit the authentication process here and assume all queries from users are authenticated by data providers through out-of-band services (e.g., xxx). We will have more query authentication discussions later (§**??**). Other roles are semi-honest: the cloud storage provider are curious about the plaintext of outsourced data, but will honestly execute queries to preserve reputation; the data users can query matching records but will learn noth-

Algorithm 1: Index Preparation

Input:
  ptx[] ← plaintexts
  m ← bit-length of bloom filters
Output:
  cid[] ← cipher-identifiers of plaintexts for point search
  brange[] ← biased range of plaintexts for range search

Function `Prepare()` **do**
  subscript[] ← ∅
  sortedPtx[] ← `sort`(*ptx*)
  ▽ **Prepare cipher-identifiers**
  foreach *plaintext p in sortedPtx* **do**
    foreach *evaluator e in filter.hashFuncSet* **do**
      subscript.`insert`(*e*(*p*))
    cid.`insert`(*subscript*)
    subscript[] ← ∅
  ▽ **Prepare biased ranges in plaintext order**
  $range_{min} \leftarrow 0$, $range_{max} \leftarrow m$
  foreach *plaintext p in sortedPtx* **do**
    $bias \leftarrow k * p + hash(p)$ ▷ $hash(p) \in [0, k)$
    $p.rmin \leftarrow bias + range_{min}$
    $p.rmax \leftarrow bias + range_{max}$
    brange.`insert`(<*p.rmin, p.rmax*>)
  return cid, brange

Algorithm 2: MBF-Tree Construction

Input from Algorithm 1:
  cid[] ← cipher-identifiers
  brange[] ← biased ranges
Output:
  MBF-Tree ← HoTEE's stealth index for ciphertext

Struct {
  range myRange ▷ *for range indexing*
  bloomfilter myFilter ▷ *for point comparison*
  HOM value ▷ *homomorphic ciphertext*
  Node fn ▷ *father node for tree traversing*
} *MBFNode*;

Function `CreateInner`($Node_i$, $Node_j$) **do**
  $(inner.left,\ inner.right) \leftarrow (Node_i,\ Node_j)$
  $(Node_i.fn,\ Node_j.fn) \leftarrow inner$
  ▽ **An inner node's range is the union of its child nodes' ranges**
  $inner.myRange \leftarrow Node_i.myRange \cup Node_j.myRange$
  return inner
Function `CreateLeaf()` **do**
  leaf[]← ∅
  for *i in range(cid.length)* **do**
    $leaf[i].myFilter \leftarrow cid[i]$
    ▽ **Leaves are linked in plaintext order**
    if *i != cid.length* **then**
      $leaf[i].next \leftarrow leaf[i+1]$
    $leaf[i].value \leftarrow HOM(ptx[i])$
  return leaf
Function `CreateMBFTree()` **do**
  le[]← `CreateLeaf`()
  in[]← ∅, $ptr \leftarrow 0$
  ▽ **Pair-wise inner node construction**
  while $ptr < le.length$ **do**
    if $ptr = le.length - 1$ **then**
      in.`insert`(`CreateInner` *(le[ptr],le[ptr-1].fn)*)
    else
      in.`insert`(`CreateInner` *(le[ptr],le[ptr+1])*)
    $ptr \leftarrow ptr + 2$ ▷ *pair-wise construction*
  ▽ **Construct MBF-Tree in a bottom-up manner**
  while $in.length > 2$ **do**
    n[]← ∅, $ptr \leftarrow 0$
    while $ptr < in.length$ **do**
      if $ptr = in.length - 1$ **then**
        n.`insert`(`CreateInner` *(in[ptr],in[ptr-1].fn)*)
      else
        n.`insert`(`CreateInner` *(in[ptr],in[ptr+1])*)
      $ptr \leftarrow ptr + 2$
    $in \leftarrow n$
  ▽ **Entry of MBF-Tree stealth index**
  if $in.length = 2$ **then**
    $root \leftarrow$ `CreateInner`($in[0], in[1]$)
  else
    $root \leftarrow in[0]$
  return root

ing else stored in the cloud storage while preserving the query privacy.

We assume the proxy server is semi-honest. The proxy server will honestly execute data lookup and query routing, but is curious about the data confidentiality and query confidentiality. Such assumption requires the HoTEE's proxy server design to reveal data locality without leaking data confidentiality, and preserve query privacy while maintaining query functionality.

6 Implementation

7 Evaluation

8 Discussion

8.1 HoTEE's Generality

9 Conclusion

References

[1] Amazon AWS. 2022. State & local government. Retrieved 10/27/2022 from https://aws.amazon.com/stateandlocal/.

[2] Edgeless Systems GmbH. 2022. *EdgelessDB Official Website. Retrieved March 1, 2022*. https://www.edgeless.systems/products/edgelessdb.

[3] Google Cloud. 2022. Harvard global health institute covid-19 forecast. Retrieved 10/27/2022 from https://cloud.google.com/blog/products/ai-machine-learning/google-cloud-covid-19-public-forecasts.

[4] HElib-KV Lookup. 2022. *HElib Country Lookup Example.* https://github.com/homenc/HElib/tree/master/examples/BGV_country_db_lookup.

[5] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. Order preserving encryption for numeric data. In *Proceedings of the 2004 ACM SIGMOD*

Algorithm 3: MBF-Tree Stealth Search

```
Input:
    root ← root node (entry) of an MBF-Tree
    key (<cid, brange>) ← stealth search target
    opt (p | l | r) ← search option: point search or range
    search with left or right boundary
Function PointSearch(root, key, opt=p) do
    ▽ Use cipher-identifier for key matching
    if isLeaf(root) then
        if key.cid = root.cid then
            return root
        else
            return null
    ▽ Use biased range to traverse MBF-Tree
    if key.brange ∈ root.left.myRange then
        return PointSearch(root.left, key)
    if key.brange ∈ root.right.myRange then
        return PointSearch(root.right, key)
    ▽ Case 1:left boundary is non-existent
    if opt = l then
        ▽ Find smallest data larger than key
        root ← root.right
        while !isLeaf(root) do
            root ← root.left
        return root
    ▽ Case 2:right boundary is non-existent
    if opt = r then
        ▽ Find largest data smaller than key
        root ← root.left
        while !isLeaf(root) do
            root ← root.right
        return root
Function RangeSearch(root, [key1, key2]) do
    value[]← ∅
    left ← PointSearch(root, key1, l)
    right ← PointSearch(root, key2, r)
    ▽ Traverse linked leaves in order
    while left.next != right do
        value.insert(left.next)
        left ← left.next
    return value
```

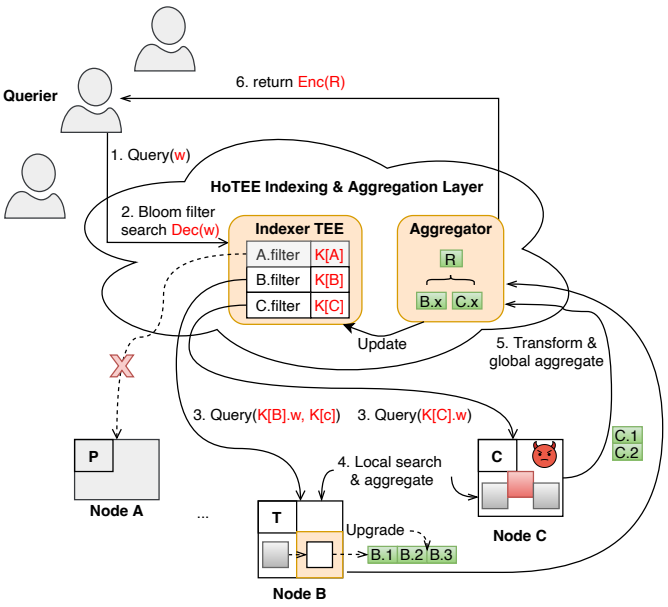


Figure 6: HoTEE architecture.

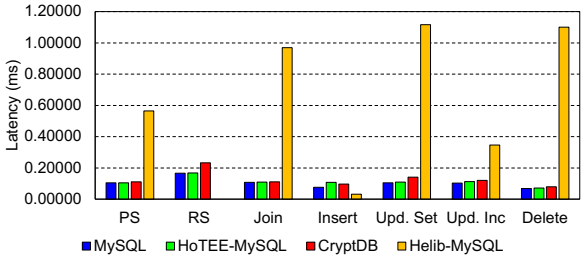


Figure 7

international conference on Management of data, pages 563–574, 2004.

[6] Panagiotis Antonopoulos, Arvind Arasu, Kunal D Singh, Ken Eguro, Nitish Gupta, Rajat Jain, Raghav Kaushik, Hanuma Kodavalla, Donald Kossmann, Nikolas Ogg, et al. Azure sql database always encrypted. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1511–1525, 2020.

[7] Arvind Arasu, Spyros Blanas, Ken Eguro, Manas Joglekar, Raghav Kaushik, Donald Kossmann, Ravi Ramamurthy, Prasang Upadhyaya, and Ramarathnam Venkatesan. Secure database-as-a-service with cipherbase. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 1033–1036, 2013.

[8] Arvind Arasu, Spyros Blanas, Ken Eguro, Raghav Kaushik, Donald Kossmann, Ravishankar Ramamurthy, and Ramarathnam Venkatesan. Orthogonal security with cipherbase. In *CIDR*, 2013.

[9] David W Archer, Dan Bogdanov, Yehuda Lindell, Liina Kamm, Kurt Nielsen, Jakob Illeborg Pagter, Nigel P Smart, and Rebecca N Wright. From keys to databases—real-world applications of secure multi-party computation. *The Computer Journal*, 61(12):1749–1771, 2018.

[10] Mihir Bellare, Marc Fischlin, Adam O'Neill, and Thomas Ristenpart. Deterministic encryption: Definitional equivalences and constructions without random oracles. In *Annual International Cryptology Conference*, pages 360–378. Springer, 2008.

[11] Steven Michael Bellovin and William R Cheswick. Privacy-enhanced searches using encrypted bloom filters. 2007.

[12] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

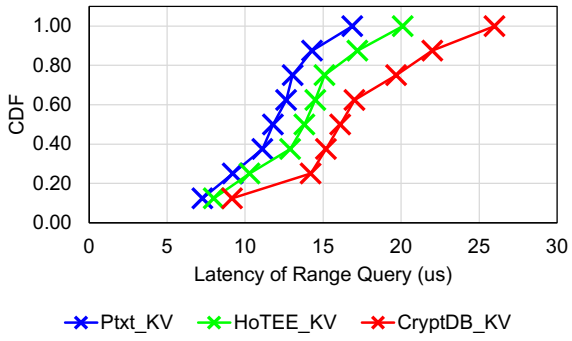


Figure 8

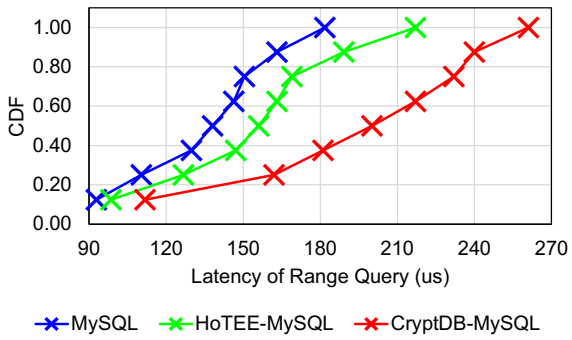


Figure 9

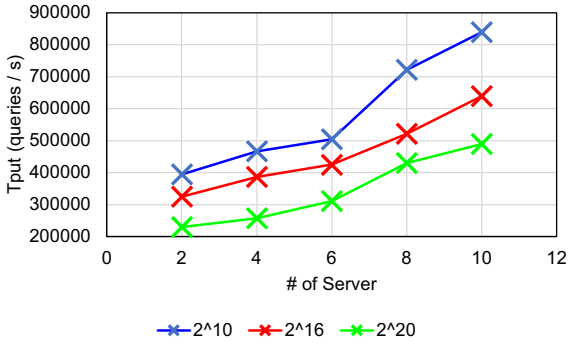


Figure 11

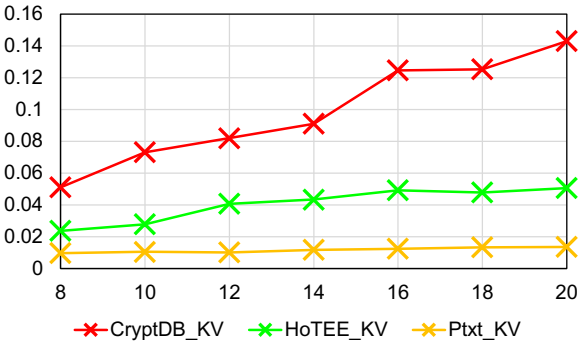


Figure 12

[13] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C Hsieh, Deborah A Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E Gruber. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, 26(2):1–26, 2008.

[14] Rupali Chopade and Vinod Pachghare. Mongodb indexing for performance improvement. In *ICT Systems and Sustainability*, pages 529–539. Springer, 2020.

[15] Victor Costan and Srinivas Devadas. Intel sgx explained. *Cryptology ePrint Archive*, 2016.

[16] Ronald Cramer, Ivan Bjerre Damgård, et al. *Secure multiparty computation*. Cambridge University Press, 2015.

[17] Emma Dauterman, Eric Feng, Ellen Luo, Raluca Ada Popa, and Ion Stoica. Dory: An encrypted search system with distributed trust. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1101–1119, 2020.

[18] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P Smart. Ring switching in bgv-style homomorphic encryption. In *International Conference on Security and Cryptography for Networks*, pages 19–37. Springer, 2012.

[19] Paul Grubbs, Thomas Ristenpart, and Vitaly Shmatikov. Why your encrypted database is not secure. In *Proceed-*

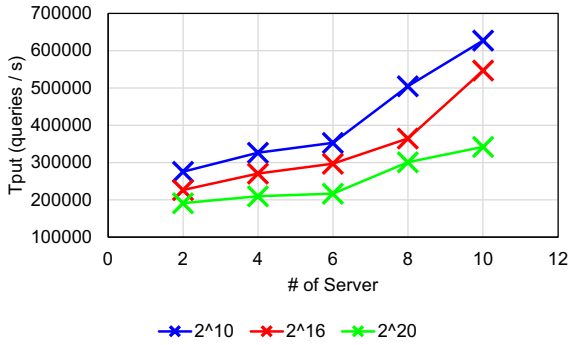


Figure 10

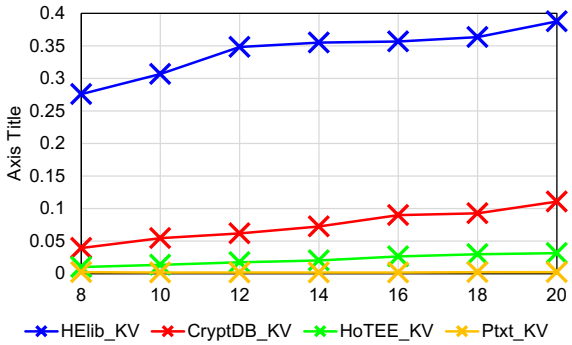


Figure 13

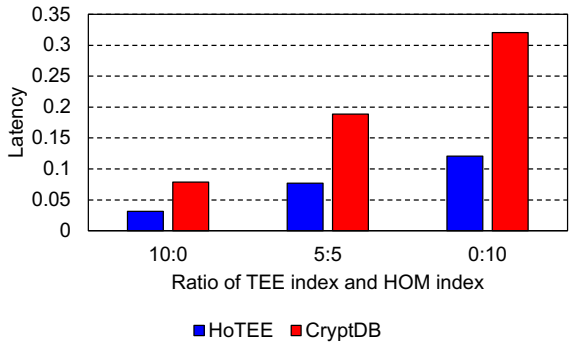


Figure 14

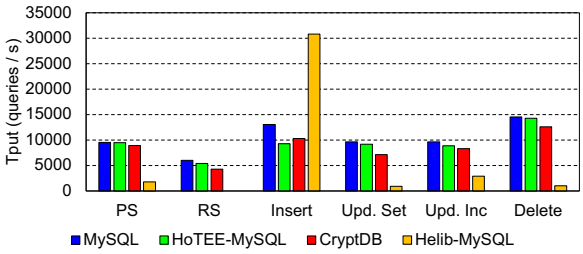


Figure 15

ings of the 16th workshop on hot topics in operating systems, pages 162–168, 2017.

[20] Shai Halevi and Victor Shoup. Algorithms in helib. In *Annual Cryptology Conference*, pages 554–571. Springer, 2014.

[21] Scott R. Intel. *SGX 2.0 (scalable SGX).* https://github.com/intel/linux-sgx/issues/899.

[22] Jianyu Jiang, Claudio Soriente, and Ghassan Karame. On the challenges of detecting side-channel attacks in sgx. In *25th International Symposium on Research in Attacks, Intrusions and Defenses*, pages 86–98, 2022.

[23] Emilia Käsper and Peter Schwabe. Faster and timing-attack resistant aes-gcm. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 1–17. Springer, 2009.

[24] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. Crypten: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems*, 34:4961–4973, 2021.

[25] Scott T Leutenegger and Daniel Dias. A modeling study of the tpc-c benchmark. *ACM Sigmod Record*, 22(2):22–31, 1993.

[26] Dong Li, Xiaofeng Liao, Tao Xiang, Jiahui Wu, and Junqing Le. Privacy-preserving self-serviced medical diagnosis scheme based on secure multi-party computation. *Computers & Security*, 90:101701, 2020.

[27] Dongxi Liu and Shenlu Wang. Programmable order-preserving secure index for encrypted database query. In *2012 IEEE Fifth International Conference on Cloud Computing*, pages 502–509. IEEE, 2012.

[28] Yanbin Lu et al. Privacy-preserving logarithmic-time search on encrypted data in cloud. In *NDSS*, 2012.

[29] Murali Mani, Kinnari Shah, and Manikanta Gunda. Enabling secure database as a service using fully homomorphic encryption: Challenges and opportunities. *arXiv preprint arXiv:1302.2654*, 2013.

[30] AB MySQL. Mysql, 2001.

[31] Proteet Paul, Tushar Gupta, and Shamik Sural. Poster: Asql-attribute based access control extension for sql. In *Proceedings of the 27th ACM on Symposium on Access Control Models and Technologies*, pages 259–261, 2022.

[32] Raluca Ada Popa, Catherine MS Redfield, Nickolai Zeldovich, and Hari Balakrishnan. Cryptdb: protecting confidentiality with encrypted query processing. In *Proceedings of the twenty-third ACM symposium on operating systems principles*, pages 85–100, 2011.

[33] Behandelt PostgreSQL. Postgresql. *Web resource: http://www. PostgreSQL. org/about*, 1996.

[34] Christian Priebe, Kapil Vaswani, and Manuel Costa. Enclavedb: A secure database using sgx. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 264–278. IEEE, 2018.

[35] Mario Schkolnick. Secondary index optimization. In *Proceedings of the 1975 ACM SIGMOD international conference on Management of data*, pages 186–192, 1975.

[36] Meng Shen, Junxian Duan, Liehuang Zhu, Jie Zhang, Xiaojiang Du, and Mohsen Guizani. Blockchain-based incentives for secure and collaborative data sharing in multiple clouds. *IEEE Journal on Selected Areas in Communications*, 38(6):1229–1241, 2020.

[37] Erez Shmueli, Ronen Vaisenberg, Ehud Gudes, and Yuval Elovici. Implementing a database encryption solution, design and implementation issues. *Computers & security*, 44:33–50, 2014.

[38] Yuanyuan Sun, Sheng Wang, Huorong Li, and Feifei Li. Building enclave-native storage engines for practical encrypted databases. *Proceedings of the VLDB Endowment*, 14(6):1019–1032, 2021.

[39] Stephen Lyle Tu, M Frans Kaashoek, Samuel R Madden, and Nickolai Zeldovich. Processing analytical queries over encrypted data. 2013.

[40] Harshavardhan Unnibhavi, David Cerdeira, Antonio Barbalace, Nuno Santos, and Pramod Bhatotia. Secure and policy-compliant query processing on heterogeneous computational storage architectures. In *ACM SIGMOD/PODS International Conference on Management of Data 2022*, 2022.

[41] Pengfei Wu, Jianting Ning, Jiamin Shen, Hongbing Wang, and Ee-Chien Chang. Hybrid trust multi-party computation with trusted execution environment.

[42] Yu Xia, Xiangyao Yu, Matthew Butrovich, Andrew Pavlo, and Srinivas Devadas. Litmus: Towards a practical database management system with verifiable acid properties and transaction correctness. In *Proceedings of the 2022 International Conference on Management of Data, Philadelphia, PA, USA*, pages 12–17, 2022.

[43] Tao Yang, Jinming Wang, Weijie Hao, Qiang Yang, and Wenhai Wang. Hybrid cloud-edge collaborative data anomaly detection in industrial sensor networks. *arXiv preprint arXiv:2204.09942*, 2022.

[44] Statistics Canada. Zachary Zanussi. *Privacy Preserving Technologies Part Two: Introduction to Homomorphic Encryption.* https://www.statcan.gc.ca/en/data-science/network/homomorphic-encryption.